Greenhouse Management System

Kaloyan Milev

Realisation

Table of Contents

Purpose and related documents....1
Realisation process....2
Development setup....2
Firmware implementation....2
Demonstrations on the assembled prototype....4
OLED feedback and the display correction....5
Data integration....6
Verification and validation....8
Software verification....8
Caretaker validation....8
Local deployment....8
Conclusion....9

Purpose and related documents

I realised a one-plant greenhouse prototype that waters dry soil automatically and makes the plant condition visible locally and in Grafana. The ESP32 controls watering; MQTT, Node.js and PostgreSQL provide the reporting and storage path. This report explains the implementation choices, integration work and results that connect the design to the demonstrated prototype.

The supporting documents provide the detailed diagrams and visual evidence:

- ***Design Presentation***: architecture on pages 3–10, wiring and power connections on pages 12–14, module responsibilities on pages 16–18, and software flow on pages 20–26.
- ***Realization PDF***: annotated OLED on page 2, dashboard on page 3, assembled prototype on page 4, and demonstration slides for temperature and humidity, light and reservoir sensing on pages 5–7, Irrigation example on page 8.

The detailed architecture on page 4 of the design presentation matches the implemented reporting path: Node.js consumes MQTT messages and inserts records, while Grafana queries PostgreSQL directly. The earlier diagrams on pages 3, 7 and 9 include remote commands and settings. These are design proposals; the completed firmware uses fixed thresholds and does not subscribe to a command topic.

Code references use paths relative to the Greenhouse project folder.

Realisation process

Development setup

I wrote the firmware in C with ESP-IDF and used its GPIO, ADC, Wi-Fi and FreeRTOS facilities. The dependency lock records ESP-IDF 6.0.3, DHT 1.2.0, ESP-MQTT 1.1.0 and a pinned SSD1306 driver revision. MQTT.js and node-postgres provide the backend connections. The source-file responsibilities and call relationships are documented on design pages 16–18.

The prototype combines the ESP32, DHT11, photoresistor module, reservoir sensor, SSD1306 OLED and M5Stack watering unit. Design pages 12–14 distinguish signal wiring from power: GPIO16 is the pump-enable signal, while the watering unit receives external 5 V power; the ESP32 uses USB and the other listed modules use 3.3 V. Their common ground gives the control signal a shared electrical reference. Realization slide 4 shows the assembled reservoir, tubing, plant, controller and dashboard together.

The implementation separates the local control task from the reporting services. Sensor results are collected in one LatestReadings structure containing values, validity flags and a timestamp. Control, display and telemetry use this shared representation, so each part can distinguish an unavailable measurement from a valid value. This also keeps watering independent of Grafana: changing a dashboard time range or losing the connection to the host does not change the local thresholds.

Firmware implementation

Sensor calibration

The sensor design needed concrete calibration values before it could make useful watering decisions. I implemented a rounded average of 32 soil ADC samples and mapped raw values of 1880 and 1620 to the dry and wet endpoints. The resulting percentage is relative to this calibration; it is not a measured soil-water content.

For example, a mean raw reading of 1750 gives 100 × (1880 − 1750) / (1880 − 1620) = 50%. A reading of 1900 would give a negative result, so the conversion clamps it to 0%. Both are valid ADC readings. If an ADC call fails, the measurement is invalid and telemetry sends null instead. This detects a reported read failure; it does not detect every possible disconnected-sensor voltage.

Reservoir ADC values from 0 through 200 mean low water; 201–1700 mean sufficient water. Values outside 0–1700 or a failed read are invalid. For light, raw 2400 maps to 0% and raw 0 to 100%, with the result clamped to that range. These are relative readings, not reservoir volume or illuminance in lux.

A calibration routine remains available in the source as commented code. It keeps the pump OFF and prints the soil sample count, average, minimum and maximum. This provides a practical way to reconsider the endpoints after changing the sensor or soil. Averaging reduces short-term variation, but it cannot correct a consistently wrong reading or establish a universal moisture percentage. The endpoints therefore belong to this prototype and need checking when its physical setup changes.

```
//ADC
#define LIGHT_ADC_CHANNEL ADC_CHANNEL_6
#define WATER_LEVEL_ADC_CHANNEL ADC_CHANNEL_7
#define MOISTURE_ADC_CHANNEL ADC_CHANNEL_5

#define ADC_LOWER_BOUND 0
#define ADC_UPPER_BOUND 4095

#define MOISTURE_DRY_ADC 1880.0f
#define MOISTURE_WET_ADC 1620.0f

//Lower Bounds
#define MARK_LIGHT_LOW 0 //tested
#define MARK_WATER_LOW 200 //tested
#define MARK_MOISTURE_DRY 1880 //tetsed

//Upper Bounds
#define MARK_LIGHT_HIGH 2400 //tested
#define MARK_WATER_MAX 1700 //tested
#define MARK_MOISTURE_WET 1700 //tested
```

soil calibration

```
        if(readings->moist_valid){

                float percentage = 100.0f * (MOISTURE_DRY_ADC - readings->moisture) / (MOISTURE_DRY_ADC -
MOISTURE_WET_ADC);

                readings->moisture_pct = fminf(100.0f, fmaxf(0.0f, percentage));



                readings->activate_pump = readings->moist_valid && readings->water_valid && !readings->wat
er_low && readings->moisture >= MARK_MOISTURE_DRY;//Higher readings are drier soil

        }else {
                readings->moisture_pct = NAN;
                readings->activate_pump = false;
        }
```

percentage conversion

Watering thresholds and timing

I implemented the watering decision with separate start and stop thresholds. Valid soil readings of at least 1880 can start watering when the reservoir is sufficient and the cooldown has finished. Watering stops at a soil reading of 1700 or below, on low water or invalid control inputs, or when the five-second time limit is reached. A ten-minute cooldown follows each stop.

In a pump test, I collected 250 ml of water over 60 seconds. Dividing the measured volume by the time gives an average flow of approximately 4.17 ml/s. At this rate, a full five-second cycle would deliver about 21 ml; an earlier stop would reduce the amount.

Separate start and stop thresholds prevent small fluctuations from repeatedly switching the pump, and the cooldown allows water to spread. While watering, the loop rechecks the deadline, reads the analogue sensors without another DHT request, checks stop conditions again and delays 50 ms only if the pump remains ON. The two-second delay is reached after the pump is OFF. It is a delay between passes, not a guarantee of one telemetry message exactly every two seconds. Design page 22 shows the main-loop sequence.

At startup, the firmware sets the pump output LOW. Soil and reservoir validity directly gate watering; temperature, humidity and light faults are reported but do not independently prohibit a cycle with valid control inputs. The cooldown is held in memory, so restarting the ESP32 resets it. This matters when interpreting repeated cycles after a reboot: the ten-minute restriction applies within a running session and is not stored permanently.

As a worked example, soil ADC 1900 and reservoir ADC 800 permit a start when no cooldown is active. If soil falls to 1750, watering continues: crossing below the start threshold is not the stop condition. At 1690 it stops, even though the OLED labels the soil OK rather than WET. If the reservoir drops to 200 first, low_water stops the cycle. A dry reading immediately afterwards cannot restart it during the cooldown.

The OLED WET label uses 1620, whereas watering stops at 1700. Consequently, OK after watering is consistent with the stop condition. The photograph on realization slide 2 shows Water: LOW; the current display code writes Water ADC followed by the raw value. LOW WATER and REFILL RESERVOIR remain the warning and action text. The photograph documents an earlier label format, not a different low-water threshold.

Demonstrations on the assembled prototype

The videos embedded in ***Realization PDF*** connect the sensor inputs to visible behaviour.

Temperature and humidity, slide 5: the dashboard humidity changes from 72% to 82% during the demonstration, and the above-60% warning is visible. This shows a changing DHT11 reading reaching storage and the dashboard. The history also contains gaps, which the narration identifies as disconnections.

Light, slide 6: shading the photoresistor reduces its reported percentage towards zero; illuminating it with a flashlight raises it. The OLED and dashboard expose the same relative conversion. This demonstrates response to changing light, without establishing a calibrated lux measurement.

Reservoir, slide 7: placing the sensor in water changes the dashboard reservoir indication from LOW to OK; removing it returns the indication to LOW. The brief return delay includes residual water on the sensor, sampling and dashboard refresh. This is threshold detection rather than a measurement of litres remaining.

Irrigation, slide 8: the recording begins with dry soil, no active cooldown and insufficient reservoir water, so watering remains blocked. After water is added, irrigation begins; the later OLED view shows the soil as OK after the cycle stops. This demonstrates how reservoir availability enables a cycle. The stored stop reason distinguishes a soil-triggered stop from the time limit; the video alone does not measure the delivered dose.

OLED feedback and the display correction

Verification exposed a mismatch between pump operation and the local display. The earlier loop updated the OLED before starting a watering cycle and again after it finished, leaving the displayed pump status unchanged during watering. The pump could therefore operate while the caretaker still saw the previous OFF state. The review notes record this as a failed display requirement before the correction.

I corrected the loop by updating the display immediately after a cycle starts, before entering the frequent watering checks. The existing update after the cycle remains, making the OFF state visible again. Updating once at entry avoids rewriting the complete OLED on every short control-loop iteration. Warning priority is preserved: an air-sensor fault can show SENSOR FAULT alongside Pump ON, because the valid soil and reservoir readings still permit watering.

The recorded retest captures display output while the pump GPIO is ON and after it turns OFF. It confirms ON/WATERING and OFF/MONITORING for a normal cycle, plus the sensor-fault variant. The current source also passes these existing host scenarios. Unlike the simplified main-loop diagram on design page 22, the implementation explicitly refreshes the OLED when the pump starts. OLED communication can still delay the next stop check.

```
709                 //check if should wter
710                 updateWatering(&readings, &pump_on);
711
712                 if(pump_on){
713                         //show if the pump is on
714                         displayResults(&oled, &readings);
715                 }
716
```

.

```
743             //display results on the oled again, this time with pump off and new measurements
744             displayResults(&oled, &readings);
745
```

screenshots of the OLED updates when watering starts and stops

Data integration

Transferring measurements

Wi-Fi setup returns without waiting for an IP address, and the main loop does not call networkWaitReady(). A disconnection schedules a connection attempt after three seconds; obtaining an IP stops that retry timer. MQTT startup success sets mqtt_started, but this means the client has started, not that the broker has acknowledged a message. These distinctions implement the independent local-control path shown on design pages 17 and 23–24.

The firmware formats bounded JSON and enqueues telemetry and watering events for MQTT delivery at QoS 1, without retaining those messages. Queueing keeps delivery separate from the pump decision; a failed enqueue does not prevent a stop. Both streams share a 4096-byte RAM outbox, so a long outage can exhaust it. On greenhouse/01/status, the client publishes retained online status and configures retained offline as its last will. As design page 26 shows, the backend does not consume that status topic; Grafana uses reading age instead.

The backend accepts only its telemetry and watering topics, parses a JSON object and validates fields before parameterized INSERT statements. Telemetry does not contain device_id: saveReading() assigns 01 for the subscribed greenhouse/01/telemetry topic. Watering events do contain device_id and must match 01. Numeric measurements must be finite and in range or null. For example, humidity_pct: 101 is rejected, while humidity_pct: null is stored as an unavailable reading. A rejected message creates no reading row.

Storing watering events

The main loop queues periodic telemetry after watering has finished, so its pump_on field is normally false even when a watering cycle has just occurred. Separate start and stop events preserve those transitions in watering_events. Each event has a boot ID and cycle number. Combining device ID, boot ID, cycle number and event type gives the backend a stable identity for ignoring repeat delivery.

Start events use the dry_soil reason and a null duration; stop events include elapsed time and a supported stop reason. The message-routing logic is shown on page 25 of the design presentation.

Illustrative event pair: device 01, boot ID 1212121212121212 and cycle 3 report started at uptime 12000 ms, duration null, reason dry_soil. A stopped event at 17000 ms has duration 5000 and reason time_limit. Receiving that stop twice still leaves one stop row because all four uniqueness fields match. After a reboot, a new boot ID separates the restarted cycle counter from earlier cycles. Duplicate telemetry is not suppressed by this event-specific key.

```
You are now connected to database "greenhouse" as user "postgres".
Pager usage is off.
Null display is "NULL".
greenhouse=# WITH latest_cycle AS (
    SELECT device_id, boot_id, cycle_id
    FROM watering_events
    WHERE device_id = '01' AND is_test = false
    GROUP BY device_id, boot_id, cycle_id
    HAVING bool_or(event = 'started')
       AND bool_or(event = 'stopped')
    ORDER BY max(received_at) DESC
    LIMIT 1
)
SELECT
    w.device_id,
    w.boot_id,
    w.cycle_id,
    w.event,
    w.duration_ms,
    w.reason
FROM watering_events AS w
JOIN latest_cycle USING (device_id, boot_id, cycle_id)
WHERE w.is_test = false
ORDER BY w.occurred_at_ms, w.event;
 device_id |     boot_id      | cycle_id |  event  | duration_ms |   reason
-----------+------------------+----------+---------+-------------+------------
 01        | 35d899a3eb1a4e10 |        1 | started |        NULL | dry_soil
 01        | 35d899a3eb1a4e10 |        1 | stopped |        5002 | time_limit
(2 rows)

greenhouse=#
```

PostgreSQL screenshot of a matching watering start/stop pair, showing duration and stop reason

Interpreting dashboard data

I separated Test and Real records with is_test. The checked-in dashboard JSON defaults to Test, while the screenshot on realization slide 3 is already in Real mode. Firmware messages contain test: false, so Real must be selected to view them. The dashboard has 14 panels: six current values, reading age, latest sample details, current alerts, four history charts and watering history. Its five-second refresh affects when a stored change becomes visible.

Current-value cards select the latest stored row independently of the chart time range; history charts filter database receipt time. Device timestamps are uptime, not calendar time. If a queued reading arrives late, its recent receipt can make Reading age look fresh even though acquisition was earlier. The pump card reports the last stored command; the watering-history rows show transitions that periodic telemetry misses. Neither proves water flow. These behaviours explain the separate current values and history shown on realization slide 3.

The Current alerts query marks data older than 30 seconds as stale and does not evaluate stale air readings as current conditions. Its inclusive air ranges are 16–27 °C and 40–60% humidity. For example, the slide 3 screenshot shows 21 °C as OK and 79% humidity as HIGH, alongside LOW WATER. These air-range warnings are calculated in Grafana. The OLED shows sensor faults, low water and operating state; it does not implement those air-range warnings. Light has monitoring and history but no light-range warning.

Verification and validation

Software verification

The recorded verification covers 15 firmware host scenarios, 43 backend cases and 56 SQL assertions across the 14 dashboard panels. The existing firmware harness was also rerun successfully against the current source during this revision. It executes the application functions with controlled hardware, clock and messaging adapters. This checks software decisions and output; it does not replace measurements on the physical pump or sensors.

The cases exercise the five-second stop and ten-minute cooldown boundaries, invalid control inputs, failed MQTT enqueueing and duplicate watering events. The full cases remain in the verification report and evidence folders. The demonstrations above add visible sensor responses and an irrigation cycle, while the host scenarios check boundary conditions that are difficult to time precisely by watching a video.

Two control limitations remain. A soil ADC value of 4095 can request watering, so an in-range disconnected-sensor signal may go undetected. Stopping occurs at the next control check: a controlled slow-display scenario recorded 5010 ms against the 5000 ms target. This does not measure physical pump duration.

Caretaker validation

I passed all five caretaker tasks covering condition reading, warnings, useful watering, cycle history and local-display use during a network interruption. Dashboard updates resumed after reconnection. This self-test established qualitative acceptance for routine care, while long-term plant growth was not assessed.

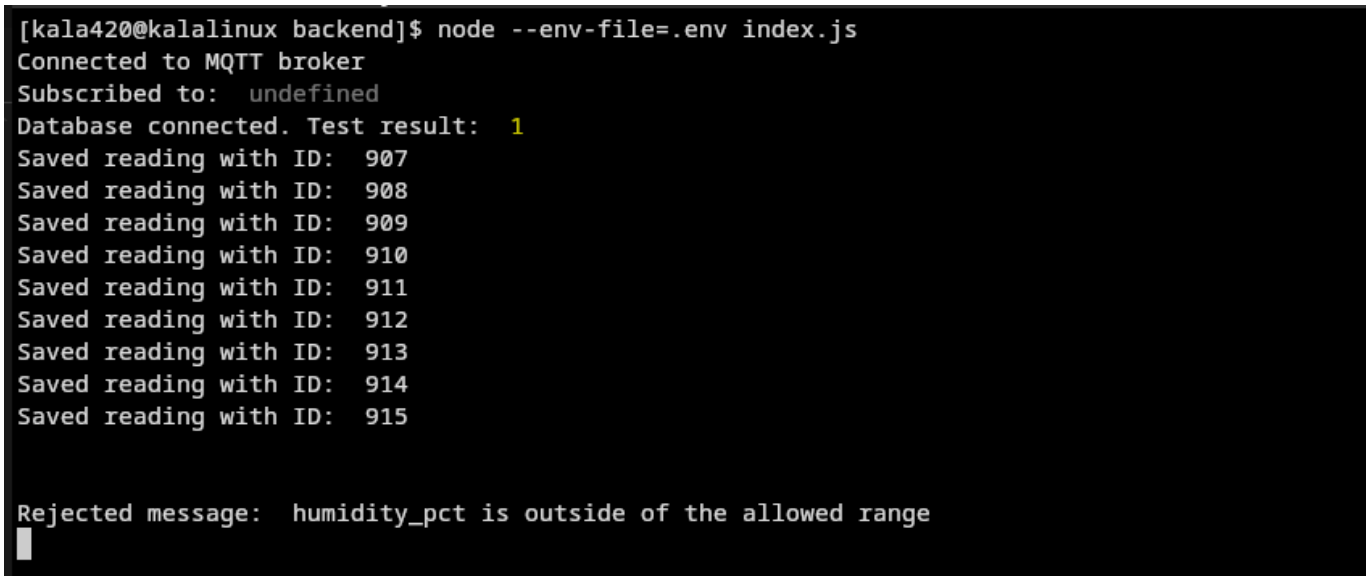
Detailed cases and acceptance findings are recorded in Greenhouse_Verification_and_Validation_Updated.docx and its evidence folders. The additional current-source run is saved in doc/validation/evidence/2026-10-28/realisation-document-check.txt.

Local deployment

I used the local host for Mosquitto, the Node.js backend, PostgreSQL and Grafana. The firmware and backend connect to the broker at 10.42.0.1:1883. Wi-Fi credentials are stored in a separate firmware header, and the backend uses its runtime PostgreSQL configuration. Importing greenhouse-overview.json requires selecting a Grafana data source with read access to both readings and watering_events.

To check a deployment, capture a message on greenhouse/01/telemetry, match its sampled_at_ms and values to a readings row with device_id 01 and is_test false, then select Real in Grafana. The device ID comes from the backend mapping, not a telemetry payload field. This trace separates broker delivery, database storage and dashboard filtering. Check watering_events separately for a start/stop pair; a current-value card alone does not establish that a complete cycle was recorded.

Once the local loop is running, a Wi-Fi or broker interruption does not suspend watering or OLED updates. This differs from a network-initialization error: app_main() uses ESP_ERROR_CHECK(networkInit()), so that failure aborts startup. History also depends on the host services. Failed database inserts are logged without persistent retry, and the local deployment uses unencrypted MQTT. An acknowledged MQTT publication is therefore not proof that the corresponding row was saved.



terminal screenshot showing the backend accepting valid readings and rejecting a humidity value of 101%

Conclusion

The realised prototype links sensor measurements to automatic watering, OLED feedback and stored history. Calibration and separate thresholds explain the control decisions; the sensor and irrigation demonstrations show their effect on the assembled system. Separate watering events preserve transitions that the current pump card can miss. The main remaining improvements are disconnected-sensor detection and durable recording during service failures.